

Коллекции и Разработка ПО

Программирование, 2023-2024

Терещенко В.В.
TG: @VI-Tershch
Email: vlad-tershch@yandex.ru

Вспомним типы данных

Тип объекта	Пример литерала/создания
Числа	<code>1234, 3.1415, 3+4j, 0b111, Decimal(), Fraction()</code>
Строки	<code>'spam', "Bob's", b'a\x01c', u'sp\xc4m'</code>
Списки	<code>[1, [2, 'three'], 4.5], list(range(10))</code>
Словари	<code>{'food': 'spam', 'taste': 'yum'}, dict(hours=10)</code>
Кортежи	<code>(1, 'spam', 4, 'U'), tuple('spam'), namedtuple</code>
Файлы	<code>open('eggs.txt'), open(r'C:\ham.bin', 'wb')</code>
Множества	<code>set('abc'), {'a', 'b', 'c'}</code>
Прочие основные типы	Булевские значения, сами типы, <code>None</code>
Типы программных единиц	Функции, модули, классы
Типы, связанные с реализацией	Скомпилированный код, трассировки стека

Списки

Список (list) представляет тип данных, который хранит набор или последовательность элементов. Во многих языках программирования есть аналогичная структура данных, которая называется массив.

```
#list of having only integers
```

```
a= [1,2,3,4,5,6]
```

```
#list of having only strings
```

```
b=["hello","john","reese"]
```

```
#list of having both integers and strings
```

```
c= ["hey","you",1,2,3,"go"]
```

addr	idx	массив		где-то в памяти
0x00001234	0	0x45a10000	0x000187fd	True
0x0000123c	1	0x00871300		...
0x00001244	2	0xad671211	0x00871300	Клер
0x0000124c	3	0x000187fd	0x00871321	42
0x00001254	4	0x00871321		...
			0x45a10000	Френк
				...
			0xad671211	Зоя

массив

0	Френк
1	Клер
2	Зоя



.append(42)



0	Френк
1	Клер
2	Зоя



0	Френк
1	Клер
2	Зоя
3	42
4	
5	

Списки

Для управления элементами списки имеют целый ряд методов. Некоторые из них:

- **append(item)**: добавляет элемент `item` в конец списка
- **insert(index, item)**: добавляет элемент `item` в список по индексу `index`
- **extend(items)**: добавляет набор элементов `items` в конец списка
- **remove(item)**: удаляет элемент `item`. Удаляется только первое вхождение элемента. Если элемент не найден, генерирует исключение `ValueError`
- **clear()**: удаление всех элементов из списка
- **index(item)**: возвращает индекс элемента `item`. Если элемент не найден, генерирует исключение `ValueError`
- **pop([index])**: удаляет и возвращает элемент по индексу `index`. Если индекс не передан, то просто удаляет последний элемент.
- **count(item)**: возвращает количество вхождений элемента `item` в список
- **sort([key])**: сортирует элементы. По умолчанию сортирует по возрастанию. Но с помощью параметра `key` мы можем передать функцию сортировки.
- **reverse()**: расставляет все элементы в списке в обратном порядке
- **copy()**: копирует список

Кроме того, Python предоставляет ряд встроенных функций для работы со списками:

- **len(list)**: возвращает длину списка
- **sorted(list, [key])**: возвращает отсортированный список
- **min(list)**: возвращает наименьший элемент списка
- **max(list)**: возвращает наибольший элемент списка

Кортежи

Кортежи (tuple) в Python – это те же списки за одним исключением. Кортежи неизменяемые структуры данных. Так же как списки они могут состоять из элементов разных типов, перечисленных через запятую. Кортежи заключаются в круглые, а не квадратные скобки.

```
>>> a = (10, 2.13, "square", 89, 'C')
```

```
>>> a
```

```
(10, 2.13, 'square', 89, 'C')
```

```
>>> a[0] = 11
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: 'tuple' object does not support item assignment
```

Кортежи

Вы можете не замечать, но вы используете кортежи когда:

- работаете с аргументами или параметрами (они хранятся как кортежи)
- возвращаете две или более переменных из функции
- итерируете ключи-значения в словаре
- используете форматирование строк

Словари

Словарь Python — это неупорядоченная последовательность данных, хранящихся в форме пар «ключ-значение». Словари записываются внутри фигурных скобок в форме ключ:значение. Они позволяют хранить большие объемы данных и предлагают оптимизированный способ для их извлечения.

```
a = {1:"first name",2:"last name", "age":33}
```

```
#print value having key=1
```

```
print(a[1])
```

```
#print value having key=2
```

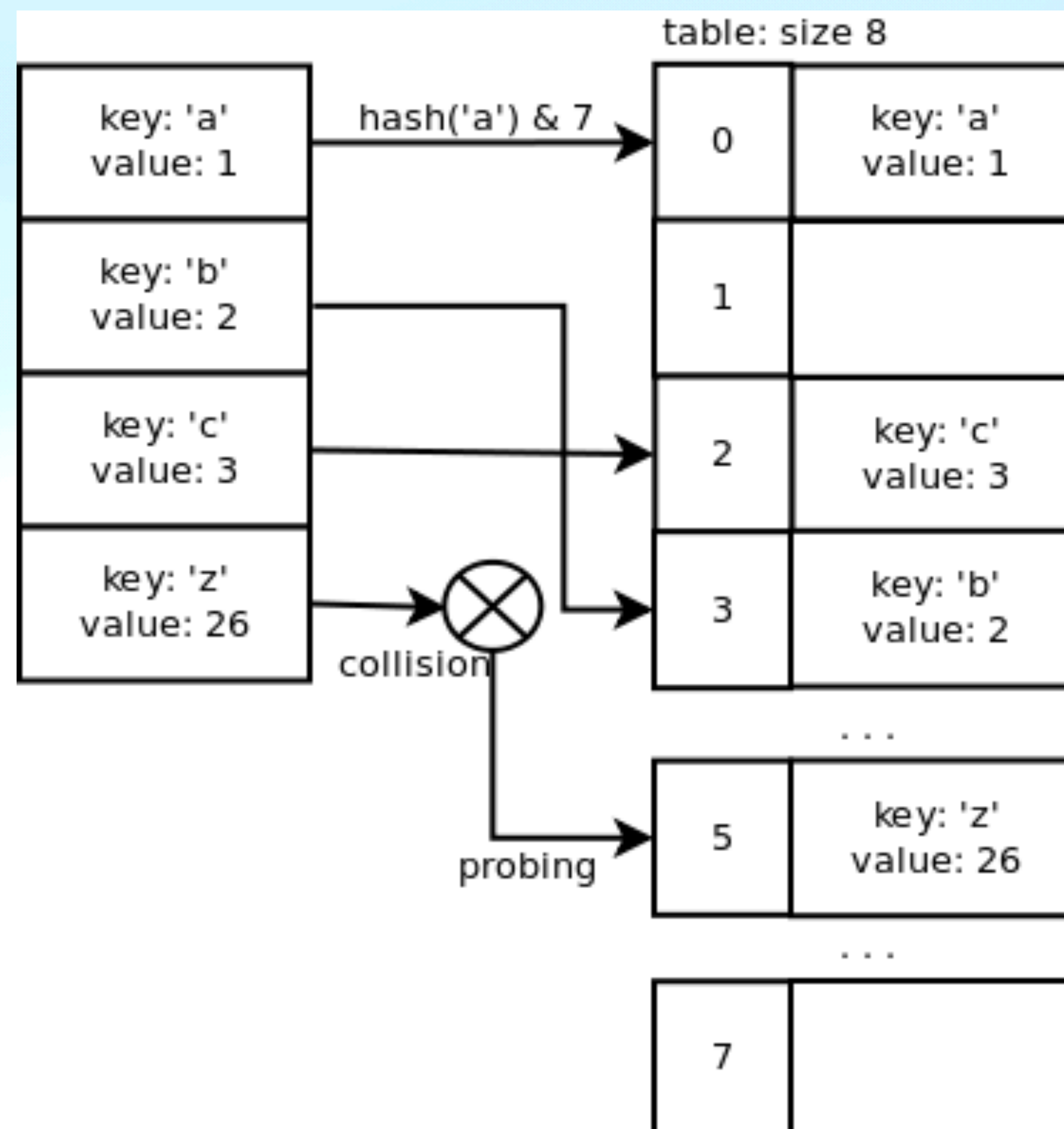
```
print(a[2])
```

```
#print value having key=«age»
```

```
print(a["age"])
```

Словари

Открытая адресация – это метод разрешения коллизий, в котором используется пробирование. В этом случае в ячейки помещаются не указатели на списки, а сами пары ключ-значение.



Множества

Множество — это набор, содержащий неупорядоченные уникальные значения. Одно множество может содержать значения любых типов.

```
color_list = ["red", "green", "green", "blue", "purple", "purple"]
```

```
color_set = set(color_list)
```

```
print(color_set)
```

Вывод (порядок может быть другим):

```
{"red", "purple", "blue", "green"}
```



Множества

$$\{\text{blue bear}, \text{pink pig}, \text{cat}\} = \{\text{pink pig}, \text{cat}, \text{blue bear}\}$$

$$\{\text{blue bird}, \text{pink pig}\} \cap \{\text{green pig}, \text{blue bear}\} = \emptyset$$

$$\{\text{yellow duck}, \text{pink pig}\} \subset \{\text{pink pig}, \text{cat}, \text{yellow duck}\}$$

$$\{\text{blue bear}, \text{pink pig}, \text{blue bird}\} - \{\text{blue bird}, \text{cow}, \text{grey duck}, \text{cat}\} = \{\text{blue bear}, \text{pink pig}\}$$

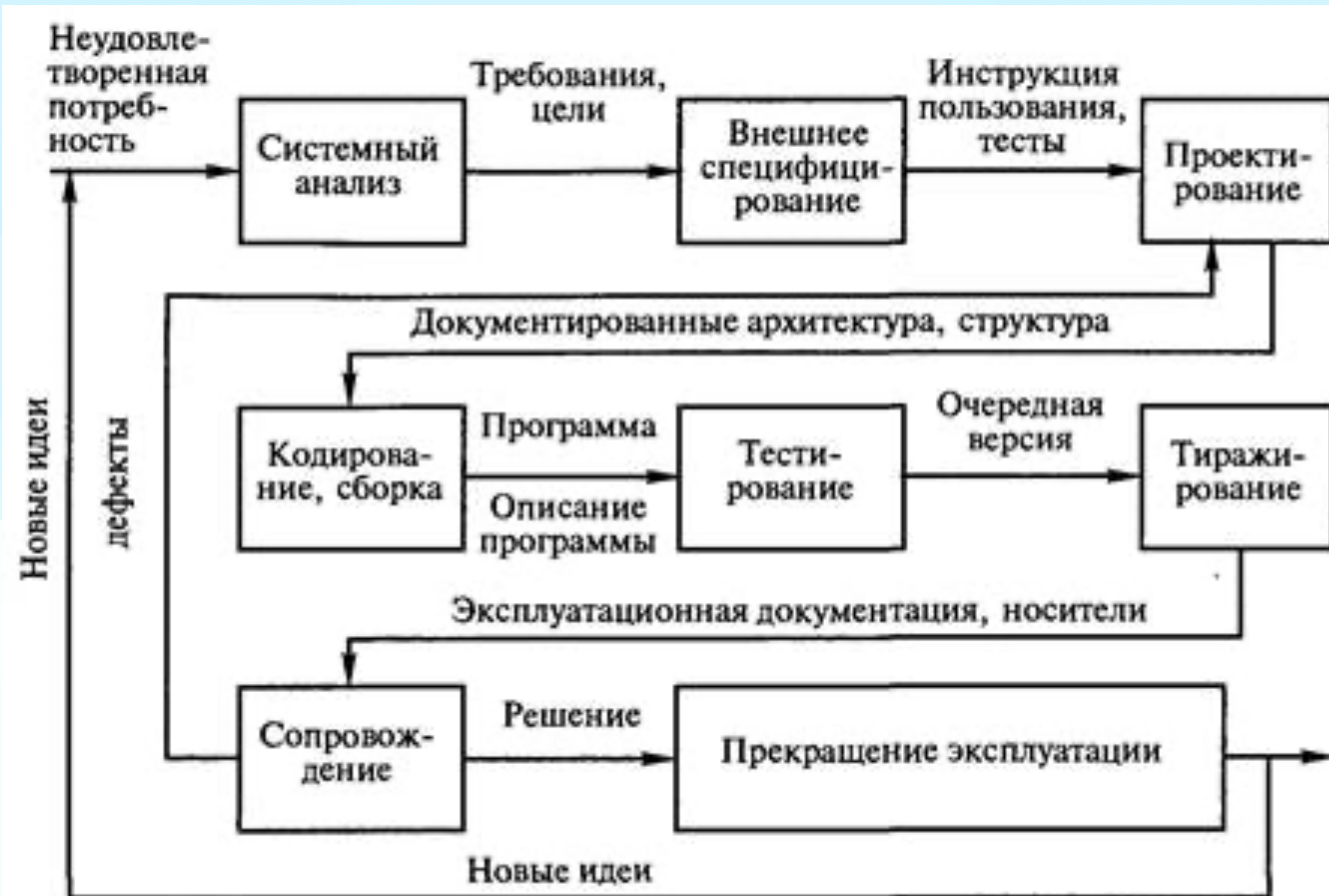
$$\{\text{light blue cow}, \text{blue bird}, \text{pink pig}\} \cup \{\text{blue bird}, \text{pink pig}, \text{brown bear}\} = \{\text{light blue cow}, \text{blue bird}, \text{pink pig}, \text{brown bear}\}$$

$$\{\text{pink pig}, \text{blue bird}, \text{light blue duck}\} \cap \{\text{light blue duck}, \text{blue bird}, \text{light blue cow}\} = \{\text{blue bird}, \text{light blue duck}\}$$

	List/Tuple	Dict	Set/FrozenSet
Get	$O(1) / O(1)$	$O(1) / O(n)$	$O(1) / O(n)$
Insert	$O(n) / O(n)$	$O(1) / O(n)$	$O(1) / O(n)$
Remove	$O(n) / O(n)$	$O(1) / O(n)$	$O(1) / O(n)$

Разработка ПО

- Разработка – это не только про написание кода
- Разработка включает в себя:
 - Анализ и проработку требований (а также подготовку всевозможной документации)
 - Работу с инфраструктурой
 - Написание кода
 - Тестирование
 - Внедрение
 - Не может быть, управление)))
 - И многое другое
- Успех проекта в синергии всех функциональных ролей



Чем живут разработчики?

- Необходимо постоянно развиваться
- Держать руку на пульсе – быть в тренде
- Обучения, экзамены, конференции – путь к получению новых знаний



Петя



Вася



Антон



Что может пойти не так?



Нас спасет

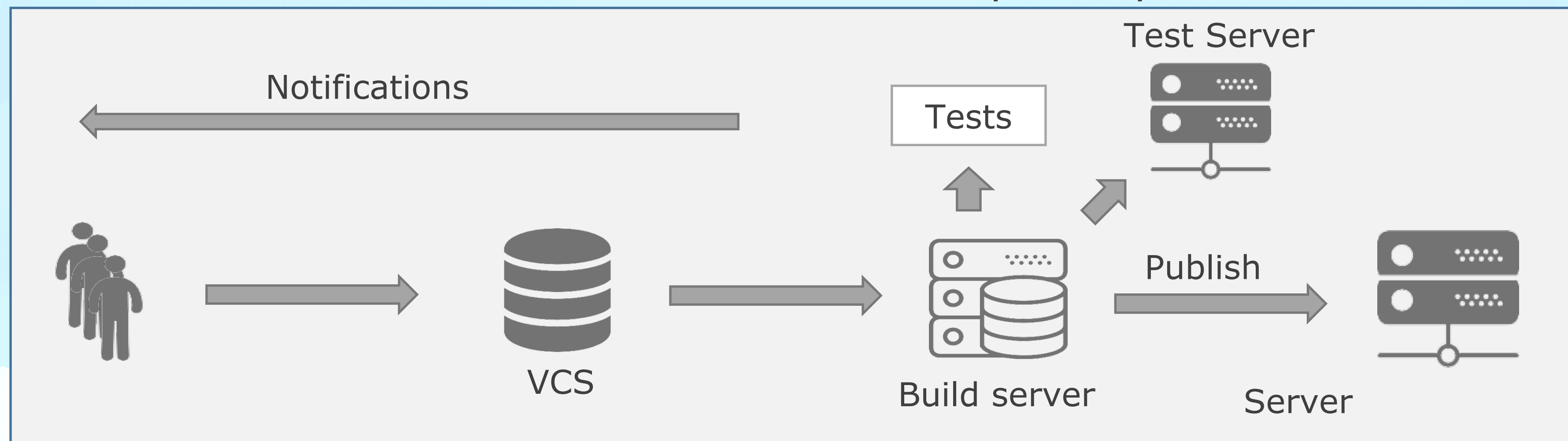


Continuous Integration

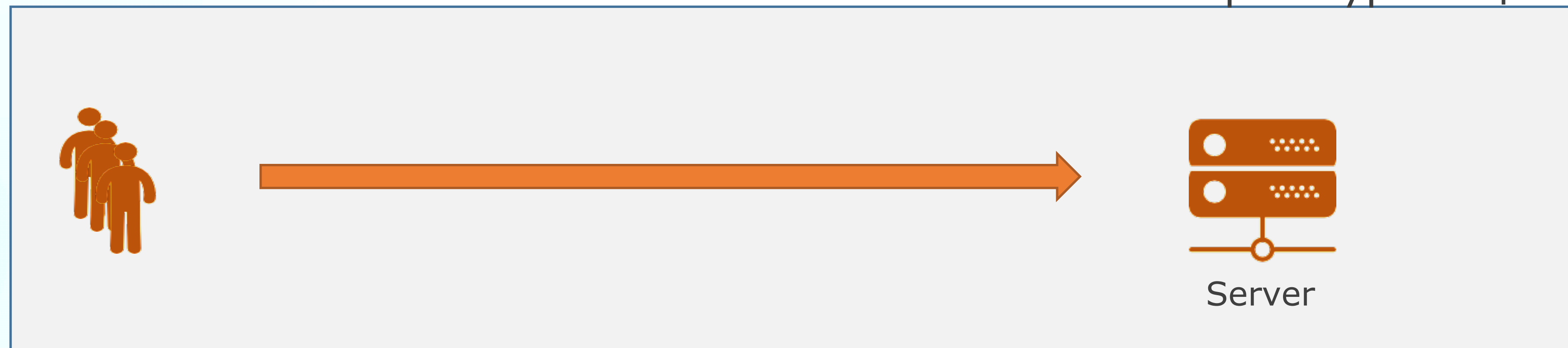
- Единый репозиторий для хранения исходного кода
- Фазы сборки:
 - Сборка проекта (еще один сложный процесс)
 - Автоматизированное выполнение тестов
 - Доставка*
- Прочие интеграции: статический анализ исходного кода, уведомления, взаимодействия с баг-трекингом и т.д.

Continuous Integration

Сборка нормального человека



Сборка курильщика



Плюсы CI

- проблемы интеграции выявляются достаточно быстро
- немедленное выполнение тестов для свежих изменений
- постоянное наличие текущей стабильной версии вместе с артефактами сборки
- команда приучается работать итеративно
- устраняет синдром “It works on my machine”

Инструменты CI



Jenkins



GitLab

Continuous Delivery & Deployment

Практика непрерывного развертывания позволяет быстрее и эффективнее передавать ПО в эксплуатацию.

«Наивысшим приоритетом для нас является удовлетворение потребностей заказчика благодаря регулярной и ранней поставке ценного программного обеспечения»

(с) манифест гибкой разработки ПО

Continuous Delivery & Deployment

CONTINUOUS DELIVERY



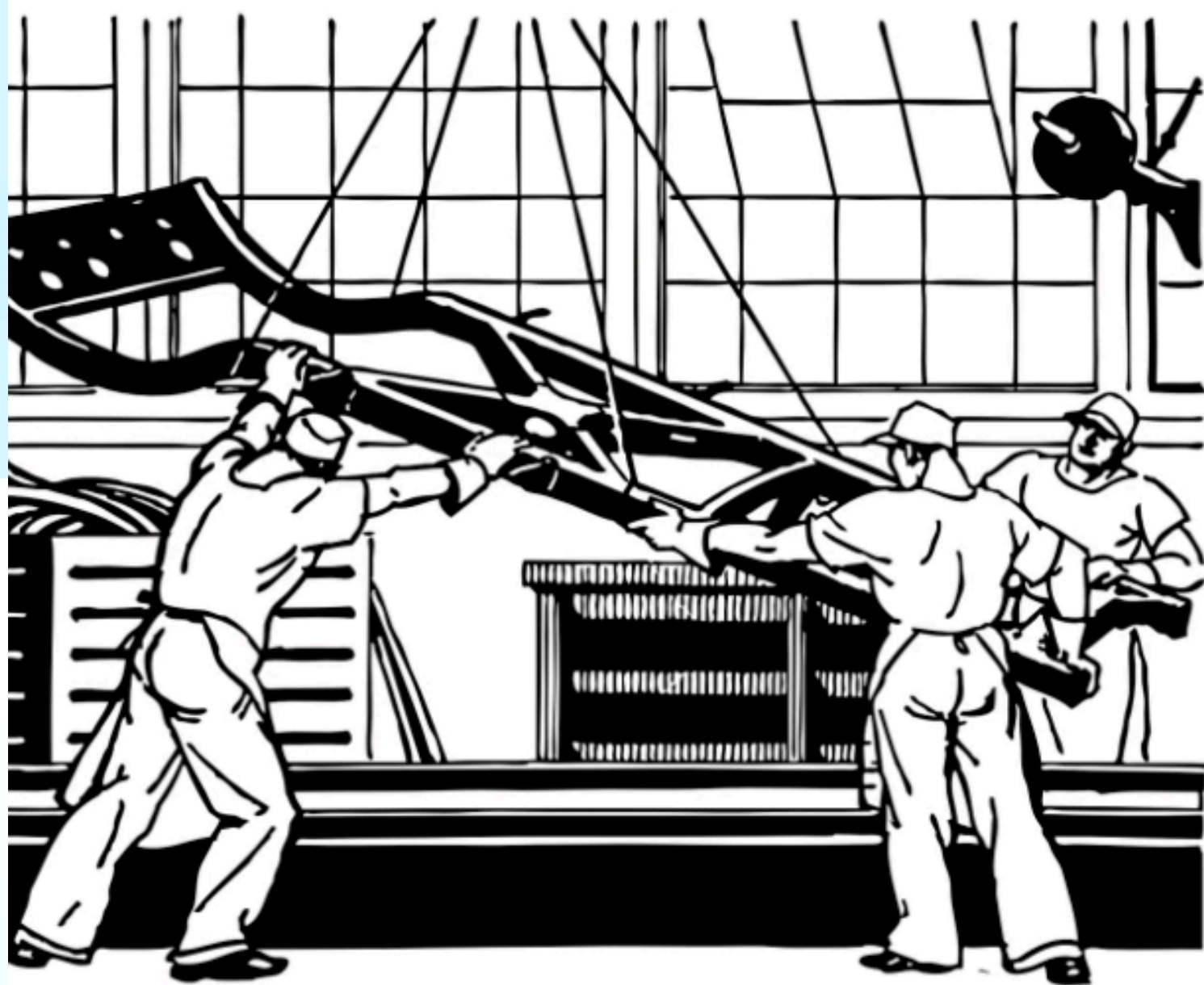
CONTINUOUS DEPLOYMENT



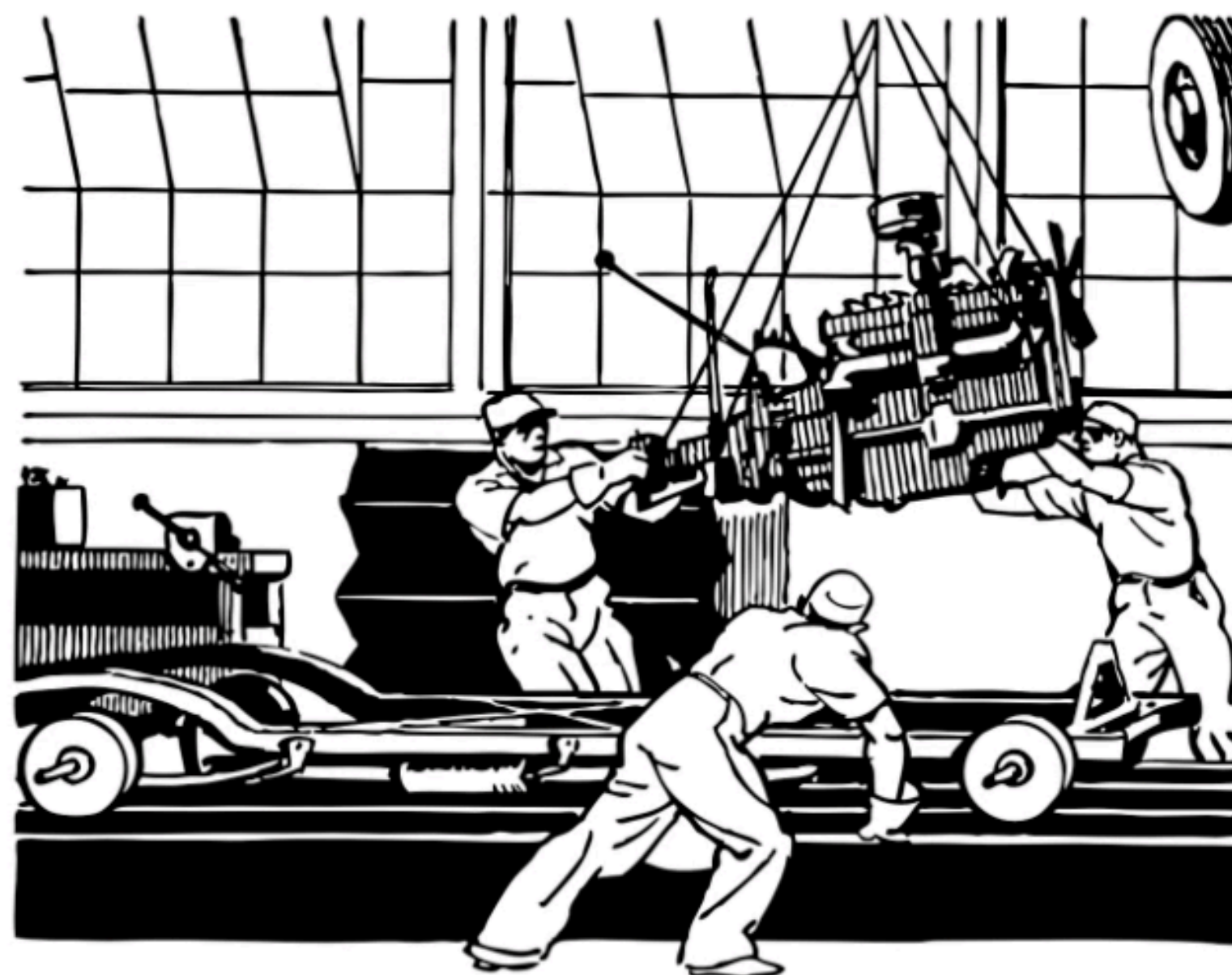
Best Practice

- КОММИТИТЬ чаще
- НЕ КОММИТИТЬ сломанный код
- все тесты должны проходить
- исправлять ошибки, как можно раньше
- наблюдать за метриками*

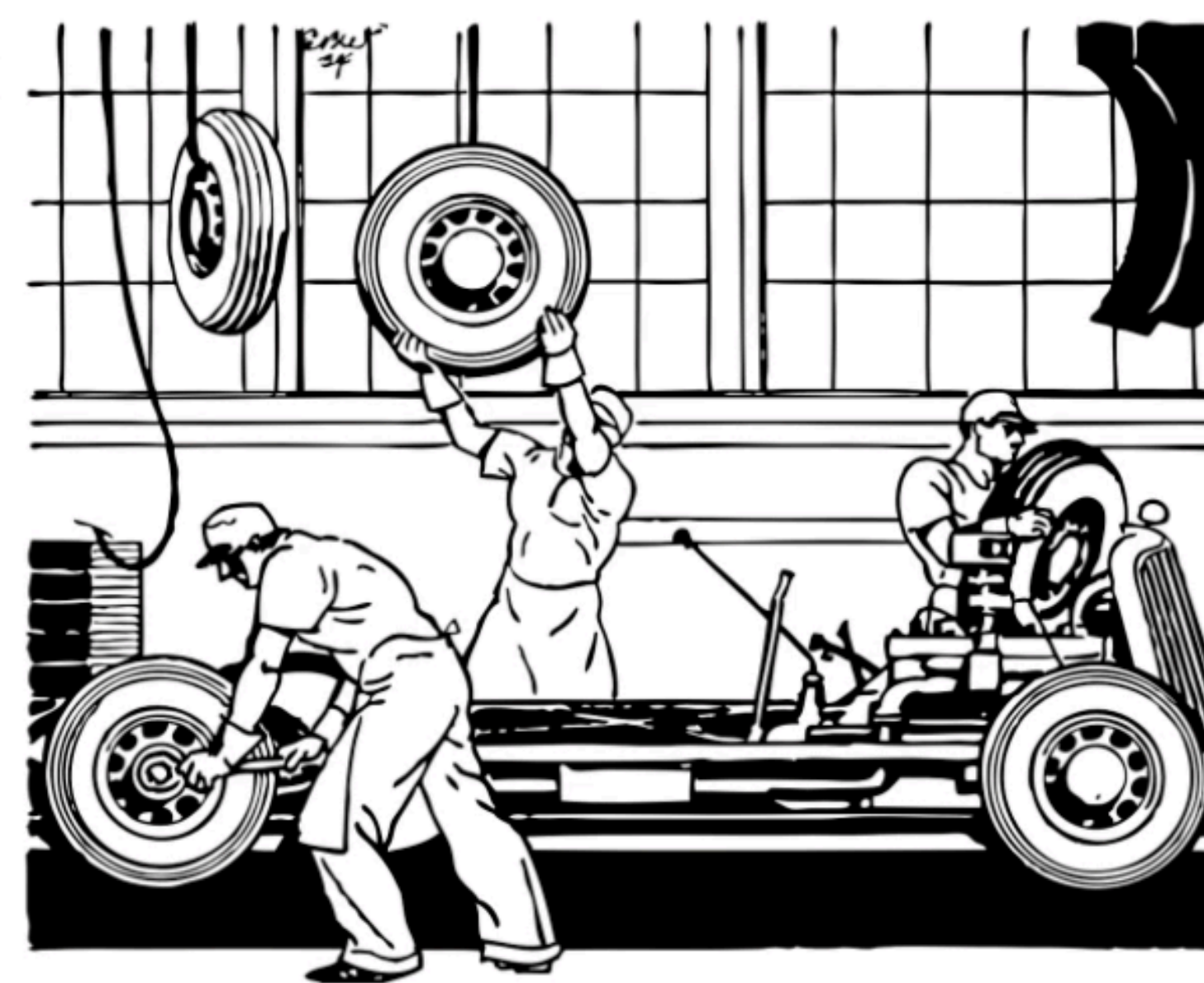
Аналогия



chassis

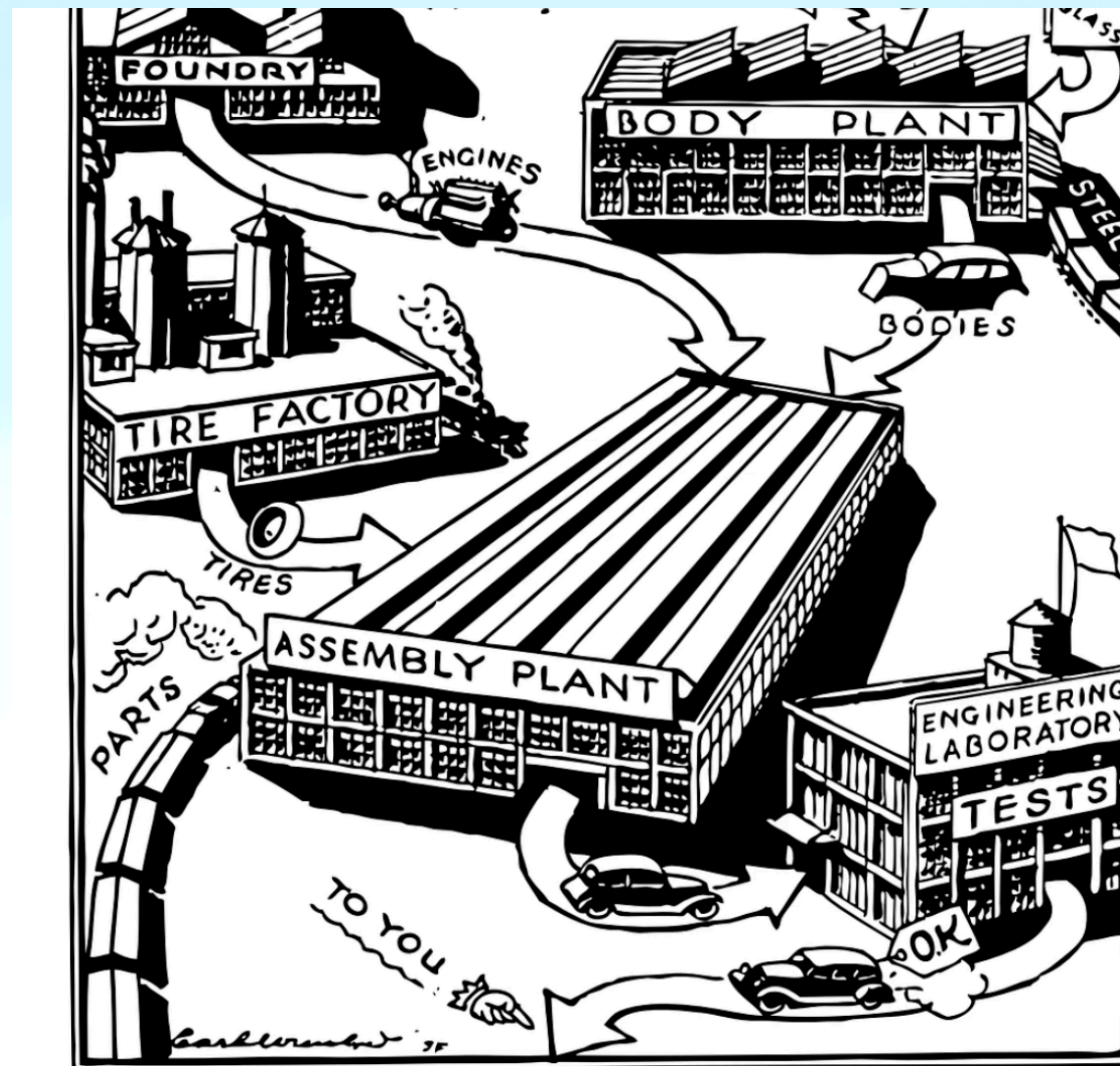


engine



wheels

Аналогия



Концепция Pipeline

Jobs

Stage

Pipeline Needs Jobs 6 Failed Jobs 1 Tests 28 Code Quality

Test

✓ secret_detecti... ↺

! sonarqube ↺

Build

✓ build ↺

Deploy

✓ deploy to dem... ↺

✓ deploy to doc... ↺

Test-e2e

✗ E2E-tests

Вопросы? 🤔